

Akshat — MLAAD, IndicSynth, Hindi clips

RTX 4060 · you own generator diversity and the clips that win the demo

Your job

IndicSynth, and the clips that decide the demo

Two jobs. Job 1 is a download that runs while you do Job 2. **Job 2 is the one only you can do**, and it is what we actually put on stage.

#	JOB	WHY
1	IndicSynth → outputs\embeddings_indicsynth	Voice-conversion spoofs, 12 Indian languages
2	Matched Hindi real/clone pairs	Our only real-world Indian evidence. We currently have zero .
3	In-the-Wild	Still a blank on the results table. Lowest priority now.

Step 0

Setup

```
cd <your Sonix folder>
.\.venv\Scripts\Activate.ps1

git fetch origin
git checkout yugal/ml-pipeline
git pull
pip install -U datasets soundfile huggingface_hub
```

That pull matters for you specifically: it contains the fix for the bug that made your In-the-Wild run report `succeeded=0 skipped(bad)=31779` five times. That was never your fault — ffmpeg was missing from your PATH and the loader had no fallback, so every file failed identically. It now falls back to soundfile.

```
Test-Path prep_indicsynth.py, src\extract_embeddings.py, stamp_labels.py
```

All three must be `True`.

Job 1

IndicSynth — start the download, then walk away

12 Indian languages: Bengali, Gujarati, Hindi, Kannada, Malayalam, Marathi, Odia, Punjabi, Sanskrit, Tamil, Telugu, Urdu. Its generator is `freevc24` — voice conversion, which is a different family from the TTS models Navya is fetching. Two families beat twice as much of one.

```
python prep_indicsynth.py --list
```

Shows the language subsets. Then:

```
python prep_indicsynth.py --per-lang 1400 --out data\indicsynth
```

TAKE A SLICE, NOT THE WHOLE THING

Some subsets have 378,000 rows and others 34,000. Pull it whole and one language drowns the rest, and the head learns *that language* instead of learning synthesis. `--per-lang 1400` keeps the languages even. The script streams, so it only downloads what it keeps.

GATE 1 – CHECK THE SPREAD BEFORE EXTRACTING

The script prints a per-language count at the end. You want **roughly equal numbers across most of the 12 languages**. If only two or three produced clips, stop and tell Yugal.

Extract and stamp

```
python src\extract_embeddings.py --split train --audio-dir data\indicsynth `
  --out outputs\embeddings_indicsynth --batch 8

python stamp_labels.py --emb-dir outputs\embeddings_indicsynth\train --label 1
python stamp_labels.py --emb-dir outputs\embeddings_indicsynth\train --check
```

GATE 2 – EVERY ROW MUST READ SPOOF (1)

`--audio-dir` extraction labels **everything 0 (real)** by default, and these are all fakes. If the check shows bonafide rows, stop — training on unstamped fakes teaches the model that synthetic Hindi is genuine, which is worse than not having the data at all.

Then send Yugal the folder `outputs\embeddings_indicsynth\`, the full `--check` output, and the per-language counts.

Job 2

Matched Hindi pairs — the important one

Your 20 clips from last round are all English — every row of `clips_manifest.csv` says so. That is a large part of why the project has no Indian-language evidence at all, and why we could not catch this bug until yesterday.

What we need is Hindi, and matched. Record a person speaking, then clone *that same person* saying something else. A matched real/cloned pair of one Indian voice is the single most convincing thing we can put in front of a judge — and no dataset can give it to us.

What to collect

- **Real Hindi — 30+ clips.** Vary everything: different phones, laptops, a noisy room, a quiet room, speakerphone, earphone mic. Include short ones (2–5 s) and quiet ones — those are the cases that historically failed.
- **Cloned Hindi — 15–20 clips.** Use commercial tools (ElevenLabs, Play.ht and similar) — the same class of tool an actual scammer uses. Family-emergency and “send money now” scripts, so it matches our impact story.
- **Clone the same speakers you recorded.** This is the part that matters most. Same voice, real and fake, side by side.

- For a few, play the cloned audio out of a laptop speaker and re-record it on a phone. That is what a scam call actually sounds like over a real line.

Folder layout — the folder is the label

```
sonix_hindi\
  real\    <- genuine human recordings
  fake\    <- cloned
```

Nothing to match by hand, nothing to get wrong. Update `clips_manifest.csv` with `filename,label,language,source,device` — and put **Hindi** in the language column this time.

THESE ARE TEST DATA. DO NOT EXTRACT OR STAMP THEM.

Send the audio folders to Yugal and stop there. They are how we *prove* the fix worked; train on them and the proof evaporates. Same rule as In-the-Wild and DF21.

Job 3

In-the-Wild — only if Jobs 1 and 2 are done

```
huggingface-cli download mueller91/In-The-Wild --repo-type dataset --local-dir data\in_the_wild

python src\extract_embeddings.py --split itw --batch 8 --out outputs\embeddings `
  --itw-root data\in_the_wild
```

No `join_labels.py` needed — the extractor reads `meta.csv` directly and handles the `bona-fide` vs `bonafide` spelling difference itself. Verify both classes are present before extracting:

```
python -c "import pandas as pd,glob; f=glob.glob(r'data\in_the_wild\*\meta.csv',recursive=True)[0];
d=pd.read_csv(f); print(d.shape); print(d.iloc[:,-1].value_counts())"
```

BEFORE YOU LOOK AT THE IN-THE-WILD NUMBER

It will be far worse than our 1.49%, and that is expected, correct, and not your fault — it uses synthesis tools we never trained on. **10-30% EER is the normal, publishable result for this benchmark.** Around 40% suggests a label problem; around 0% means something leaked. Do not spend a night trying to “fix” a correct-but-ugly number — reporting it honestly is what makes us look like researchers rather than people overselling.

And never train on it. Test set only.

Report back

```
IndicSynth : ____ clips across ____ languages
              per-language counts pasted
              stamp --check output pasted in full

Hindi clips: ____ real / ____ cloned / ____ matched pairs
              (same speaker real AND cloned = a matched pair)

In-the-Wild: ____ bonafide / ____ spoof, EER ____%   (only if reached)
```

Troubleshooting

MESSAGE	FIX
can't open file 'prep_indicsynth.py'	You did not pull, or wrong folder
could not reach vdivyasharma/IndicSynth	Check network; <code>pip install -U datasets</code>
only 2–3 languages produced clips	Stop, send Yugal the <code>--list</code> output
CUDA out of memory	<code>--batch 4</code> , then <code>--batch 2</code>
check shows bonafide rows	Stop. Re-run the stamp; if it persists, tell Yugal
no .wav files directly in ...	Point <code>--audio-dir</code> at the folder that directly holds the files
Activate.ps1 cannot be loaded	<code>Set-ExecutionPolicy -Scope Process -Bypass</code>

The situation

What broke, and why

Our detector fails on Indian speech, and we now know exactly why.

Every model up to `head_robust_v2` was trained on ASVspoof 2019 — English studio speech — plus augmentations that vary *rooms*, *noise* and *phone channels*. None of them vary *language*. So on 42,082 genuine IndicVoices clips, the baseline called **56% fake** and `robust_v2` called **58.6% fake**. That is why the Modi and Bachchan clips failed.

Navya then added IndicVoices to training and Indian false alarms collapsed to **0.02%**. Real fix? No — a second bug. IndicVoices is **bonafide-only**: every one of its ~37,000 rows is labelled real. So the only thing the model could learn from Indian audio was “*Indian phonetics* → *real*”. Tested on a cloned Amitabh Bachchan clip, `head_full_ho` calls it **real**.

MODEL	REAL BACHCHAN	CLONED BACHCHAN	WHAT IT ACTUALLY LEARNED
<code>head_robust_v2</code>	fake ✗	fake ✓	“Indian audio = fake”
<code>head_full_ho</code>	real ✓	real ✗	“Indian audio = real”

Neither is detecting anything. Each gets one column right by accident.

THE ROOT CAUSE, IN ONE SENTENCE

As long as every Indian-language sample in training is labelled real, the language itself is a shortcut to “real”, and a cloned Indian voice walks straight through. No threshold, augmentation or calibration fixes this.

The fix

Put Indian languages on **both sides of the label**, from **several different synthesis families**. Then the model cannot use language or accent as a cue — it has to find real synthesis artefacts or nothing.

DATASET	SYNTHESIS FAMILY	OWNER
Indic-CodecFake	Neural audio codecs — 8 of them (DAC, Encodec, SoundStream, SpeechTokenizer, FunCodec, AudioDec, SNAC, MIMI)	Navya
MLAAD (Indic slice)	Text-to-speech — up to 140 models	Akshat
IndicSynth	Voice conversion (freercv24)	Akshat
IndicVoices <code>have it</code>	Real human speech (bonafide side)	Navya

Three *families*, not just more hours. A head trained on one generator learns that generator — which is precisely the trap we fell into with English-only ASVspoof 2019. Generator diversity is what buys generalisation.

How we train

One model, many ablations

Two questions came up, so here they are answered for everyone.

Is the new model built from scratch, or on top of the latest one?

Neither, exactly — and this matters. Our pipeline has two stages:

STAGE	COST	REUSABLE?
audio → embeddings (frozen wav2vec2 XLS-R, 300M params)	GPU, hours	Yes — forever, per dataset
embeddings → head (MLP, 262,657 params)	CPU/GPU, minutes	Cheap to redo

So we never “continue training” `head_full_ho`. We retrain a **fresh head on the union of every embedding root** — the old ones plus the new ones. That is what `--extra-emb-root` does, and it stacks (it uses `action="append"`, verified).

Fine-tuning the existing head on only the new data would be actively wrong: it would forget the English attacks it can currently catch.

One model or several?

One production model, plus four throwaway ablation heads for the results slide. Once the embeddings exist, each head takes minutes, so the ablations are nearly free — and they are what let us say “*we measured which data helped*” instead of “*we threw everything in and hoped*”.

The expensive, parallel part is extraction. That is why it is split three ways across our GPUs. Training is serial and Yugal does it.

THE RULE THAT MAKES OR BREAKS THIS

Cap clips per (language × generator). If one TTS model contributes 40,000 clips and another contributes 200, the head learns to detect the loud one. Every extraction step below has a cap flag. Use it.

What must never enter training

These are test sets. Training on any of them destroys the numbers that prove we generalise, and a judge who spotted it would be right to dismiss the project.

- **ASVspoof 2021 DF** — our cross-dataset headline
- **In-the-Wild** — the honest generalisation test, named on slide 3
- **The IndicVoices holdout** (`embeddings_indicvoices_ho`) — 12% held out by recording
- **Akshat's 20 labelled clips** and **the Bachchan pair** — our real-world evidence

Everything else — ASVspoof 2019 train, the augmented copies, IndicVoices train split, Indic-CodecFake, MLAAD, IndicSynth — is fair training data.

SECOND HARD RULE – LABEL STAMPING

`extract_embeddings.py --audio-dir` marks **everything label 0 (real)** by default. Every spoof folder must be stamped `--label 1` afterwards, and verified with `--check`. Skip it and your fakes train as genuine — a silent, catastrophic bug that nothing warns you about.

`stamp_labels.py` was broken until 2 Sept (it never wrote a label successfully). Make sure you have pulled since then, and always read the `--check` output rather than assuming.

Sections after your own tasks are identical in all three handbooks, so nobody has to read someone else's. Datasets: Indic-CodecFake (helixometry.github.io/IndicFake) · MLAAD (deepfake-total.com/mlaad) · IndicSynth (huggingface.co/datasets/vdivyasharma/IndicSynth). When in doubt, stop and ask Yugal — a wrong number is far worse than a missing one.